

How to Build LR Parsers Which Accept Incomplete Input

Gregor Snelting
Praktische Informatik
Technische Hochschule Darmstadt
Magdalenenstr. 11c, D-61 Darmstadt

Abstract. In this short note, we describe a modification to LR parsers which allows processing of incomplete input, while at the same time building of correct abstract syntax trees. This feature is very useful for language-based editors.

1 Introduction

Many language-based editing systems allow the processing of textual input, and many of such systems employ an LR(1) or LALR(1) parser to perform syntactic analysis. Usually, standard parser generators such as Yacc are used to produce a parser. As a consequence, none of the LR-based editors known to the author (e.g. CSG [RT89], Centaur [B89]) is able to process incomplete textual input¹, e.g. “IF x=”. But for reasons explained below, the author considers this feature nice and useful, and the aim of this short note is to describe a modification to LR/LALR parsers which allows incomplete input and at the same time builds correct syntax trees. The technique is rather simple, and the author is sure that the only reason why nobody else seems to have detected or used it, is because it requires a modification to the parser generator as well as to the generated parser; and only few people have Yacc source code available.

2 Benefits and Problems of Incomplete Input

In a language-based editor which also allows text editing, placeholders within the underlying abstract syntax tree may be refined by textual input. In such an editor, users typically edit a piece of text and after a while press the “analysis please” button (e.g. the return key), which starts syntactic and semantic processing. Very often users start typing and then detect that they have to think about what comes next; thus they start analysis, following the healthy scheme: “If I have to think, let the machine work, too”. If incomplete input is not accepted, the system will come up with a syntax error, thereby disturbing the user’s thinking and making him angry. Furthermore, as has also been observed by other authors [F84], the ability to handle incomplete input gives the user a command language for free: If he simply types “IF”, the system will insert an “IF”-statement with placeholders for the condition and the “THEN” and “ELSE” statements.

Upon invocation of syntactic analysis, the parser must build an abstract syntax tree from textual input. It is well known that an LL parser can build a correct tree from incomplete input – if the tree is constructed top-down, the parser will construct placeholder nodes for textually missing parts [BS86]. With an LR parser, the situation is more difficult. In contrast to LL parsers, abstract syntax trees must be constructed bottom-up. Tree nodes are built only during reductions², that is, when a syntactic

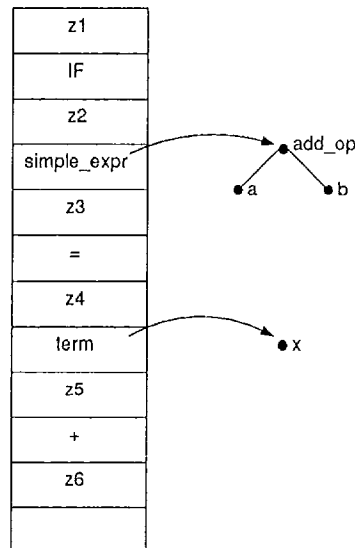
¹ We use “incomplete input” as a synonym for the more precise term “prefix of a sentential form”

² We assume the reader is familiar with LR terminology, as described e.g. in [ASU86]

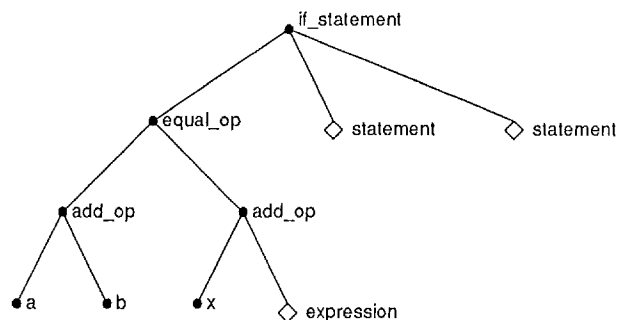
phrase has been recognized completely. During parsing, each nonterminal on the LR stack has a pointer to a tree fragment attached, which points to the subtree built during analysis of the phrase corresponding to the nonterminal. Thus, if input is incomplete, the parser stack is not completely reduced and may contain a lot of nonterminals, each of which owns a little tree fragment; but the fragments are unconnected, and since no more reductions will happen in a standard LR parser, there is no chance that the missing parent nodes will be built and the already constructed tree fragments connected to them. As an example, consider the input³

IF a+b=x+

Upon end of input, the input token is '\$' and the parser stack looks as follows:



and since $action[z_6, \$] = error$, a syntax error will be reported. We would however like the following tree to be built from the text:



where $\diamond$ denotes a placeholder of appropriate syntactic category.

³ A grammar fragment for this input is given in section 4. As usual in the LBE world, we assume that our LR parser is multiple entry (can accept all sorts of syntactic phrases, not just complete programs), and allows nonterminals in the input. Both extensions can be achieved without changing the parser generator

3 How to Modify the Parser

It is quite obvious what needs to be done: Upon end of input, instead of reporting a syntax error, additional reductions must be enforced until the “accept” state is reached. These reductions have to be “incomplete” in the sense that part of the right hand side of the reduction production is missing. The basic trick is therefore to change any LR table entry of the form

$$action[z, \$] = error$$

to a reduction entry

$$action[z, \$] = reduce \tilde{P}$$

where $\tilde{P}$ is a special “incomplete” version of a suitable production P . If we assume that the end-of-input symbol is never removed from the input, this trick will enforce additional reductions and corresponding tree building actions, until the “accept” state is reached.

Three problems remain: How do we represent incomplete reductions and tree building actions; how can the parser generator automatically select the special reduction table entries; and what happens, if the incomplete textual input is ambiguous, that is, there is more than one possibility to supplement it to a complete phrase.

3.1 Incomplete Reductions

In order to handle the first point, consider a typical production augmented with tree building action such as

$$\begin{aligned} \text{statement}_1 &\rightarrow \text{"IF" expression "THEN" statement}_2 \text{"ELSE" statement}_3 \\ \text{tree}(\text{statement}_1) &= \text{if_statement}(\text{tree}(\text{expression}), \text{tree}(\text{statement}_2), \text{tree}(\text{statement}_3)) \end{aligned}$$

An incomplete version of this production might be

$$\begin{aligned} \text{statement}_1 &\rightarrow \text{"IF" expression} \\ \text{tree}(\text{statement}_1) &= \text{if_statement}(\text{tree}(\text{expression}), \diamond, \diamond) \end{aligned}$$

or even

$$\begin{aligned} \text{statement}_1 &\rightarrow \text{"IF"} \\ \text{tree}(\text{statement}_1) &= \text{if_statement}(\diamond, \diamond, \diamond) \end{aligned}$$

where the $\diamond$ are appropriate placeholders. In general, for a production

$$X \rightarrow X_1 \dots X_n$$

each of the incomplete forms

$$\begin{aligned} X &\rightarrow X_1 \dots X_{n-1} \\ X &\rightarrow X_1 \dots X_{n-2} \\ &\dots \\ X &\rightarrow X_1 X_2 \\ X &\rightarrow X_1 \end{aligned}$$

must be considered. Such incomplete reductions can simply be characterized by the number of the original production, together with an additional integer which gives the length of the right hand side. Similarly, a tree building action with placeholders can be described by augmenting the original action with an integer number which gives the number of placeholders which have to be used instead of “real” subtrees.

We therefore add two vectors *handle_length* and *placeholder_no* to the parsing tables, which are consulted only if the current token is end-of-input. If the method to be described below has determined

that an error entry in the '\$' column of the action table should be replaced by an incomplete reduction, say

$$action[z, \$] = reduce\ X$$

where X is a production number,

$$handle_length[z]$$

gives the number of grammar symbols actually to be popped of the stack, and

$$placeholder_no[z]$$

says how many placeholders must be used instead of real subtrees; these placeholders will become the rightmost sons of the node to be constructed.

Since all incomplete reductions origin from ordinary reductions and tree construction rules, it is obvious that the incomplete reductions will build correct trees (which may contain placeholders), provided the incomplete reductions are choosen correctly.

3.2 Determining Additional Reductions

Of course, the main problem is to determine a reduction which might replace an error entry, and to compute the vectors *handle_length* and *placeholder_no*.

Let a context free grammar $G = (N, T, P, S)$ be given. During construction of the LR table, each LR state z consists of a set of LR items. Each item has one of the following forms:

$A \rightarrow B.C$	shift item
$A \rightarrow D.$	reduce item
$A \rightarrow .B$	closure item

where $B, C \in (N \cup T)^+$, $D \in (N \cup T)^*$, and $A \rightarrow BC, A \rightarrow D, A \rightarrow B \in P$. Right contexts which are used for LR and LALR items, but not for SLR items, are irrelevant for our purpose and therefore ignored⁴.

In order to determine an incomplete reduction to be used on end-of-input, we proceed as follows.

Closure items are completely ignored, since an LR state is completely determined by its shift and reduce items.

If the state contains exactly one reduce item, say $A \rightarrow D.$, this item will be used for reduction even if the end-of-input symbol '\$' is not among the legal right contexts of the item. For such a state, let $D = d_1 \dots d_j$, we therefore set

$$\begin{aligned} action[z, \$] &= reduce\ A \rightarrow D \\ handle_length[z] &= j \\ placeholder_no[z] &= 0 \end{aligned}$$

Thus, we essentially perform a standard reduction and tree-building action.

⁴ Hence, our method works equally well for LR, LALR, and SLR

If the state z contains no reduce items and exactly one shift item, say $A \rightarrow B.C$, this item will be used for an incomplete reduction. Let $B = b_1b_2...b_j$, $C = c_1c_2...c_k$,

$tree(A) = x(tree(b_{i_1}), ..., tree(b_{i_n}), tree(c_{l_1}), ..., tree(c_{l_m}))$ the tree building rule, we set

$action[z, \$] = reduce\ A \rightarrow BC$

$handle_length[z] = j$

$placeholder_no[z] = m$

If the LR state contains only closure items, it is the start state, which is a special case. For the start state, a table entry $action[z_0, \$] = error$ should not be changed, because no useful reduction can be performed on completely empty input. Instead, analysis of empty input should be avoided.

From the construction it is clear that we have the

Proposition. An LR parser augmented with incomplete reductions as described above will produce correct, but incomplete trees upon incomplete input.

3.3 Ambiguity Resolution

If there is more than one shift or reduce item in an LR state, we cannot deduce from the incomplete input which reduction to choose and which tree to build. A typical example is an isolated identifier as an incomplete statement: should it become an incomplete assignment, or should it become a procedure call?

The construction given above already employs a conflict resolution strategy: *prefer reduce items over shift items*. As we shall see in the example in section 4, this takes the end-of-input seriously and avoids making a guess what the user might have typed next.

If there is more than one shift resp. reduce item, there are two principle strategies to cope with this ambiguity: the parser can use a default reduction, or it can ask the user what to do.

The parser generator, in case it detected a conflict, might ask the grammar writer which reduction to choose on incomplete input, or it might use some brute force heuristics: if several reductions are possible, choose the one which's production occurs first in the grammar. Such a default strategy will result in an editor which makes decisions of its own if the user is not willing to supply enough input; and it might turn out that editor and user had different things in mind.

It might therefore be better to leave the "error" entry unchanged, if more than one incomplete reduction is possible. Hence, in some situations incomplete input will again produce a syntax error. This strategy has the disadvantage that the initial goal of allowing incomplete input is partially corrupted, but has the advantage that with ambiguous incomplete input, the user is forced to state explicitly what he means. Instead of just issuing a syntax error, it might be useful to generate a menu of possible completions, which is based upon the tree-building actions which are attached to the conflicting incomplete reductions.

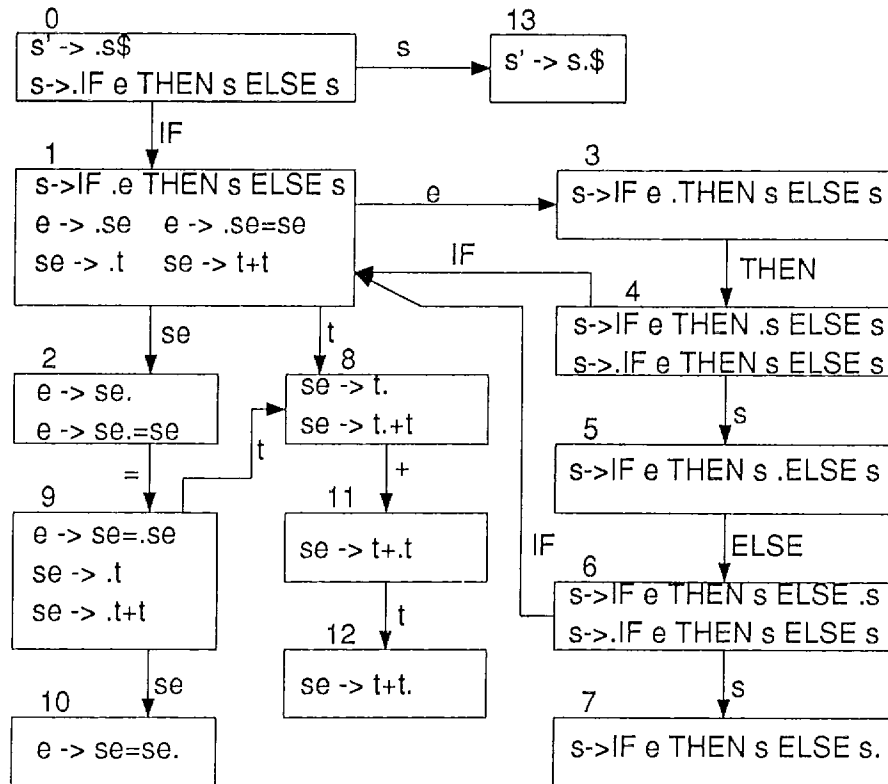
Fortunately, conflicts which cannot be solved by the "prefer reduce items" rule are rather seldom with common programming languages.

4 An Example

In conclusion of this note, we would like to present an example. Consider the grammar fragment

- (1) $\text{statement}_1 \rightarrow \text{"IF" expression "THEN" statement}_2 \text{"ELSE" statement}_3$
 $\text{tree}(\text{statement}_1) = \text{if_statement}(\text{tree}(\text{expression}), \text{tree}(\text{statement}_2), \text{tree}(\text{statement}_3))$
- (2) $\text{expression} \rightarrow \text{simpleexpr}$
 $\text{tree}(\text{expression}) = \text{tree}(\text{simpleexpr})$
- (3) $\text{expression} \rightarrow \text{simpleexpr}_1 \text{"=" simpleexpr}_2$
 $\text{tree}(\text{expression}) = \text{equal_op}(\text{tree}(\text{simpleexpr}_1), \text{tree}(\text{simpleexpr}_2))$
- (4) $\text{simpleexpr} \rightarrow \text{term}$
 $\text{tree}(\text{simpleexpr}) = \text{tree}(\text{term})$
- (5) $\text{simpleexpr} \rightarrow \text{term}_1 \text{"+" term}_2$
 $\text{tree}(\text{simpleexpr}) = \text{add_op}(\text{tree}(\text{term}_1), \text{tree}(\text{term}_2))$

The LR(0) subautomaton corresponding to this grammar fragment is⁵



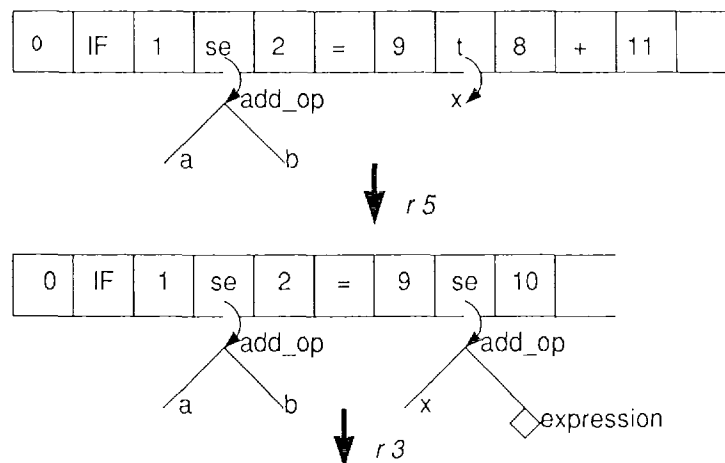
⁵ "s" stands for "statement", "e" for "expression", "se" for "simpleexpr", and "t" for "term". A "term" is considered to be a terminal symbol

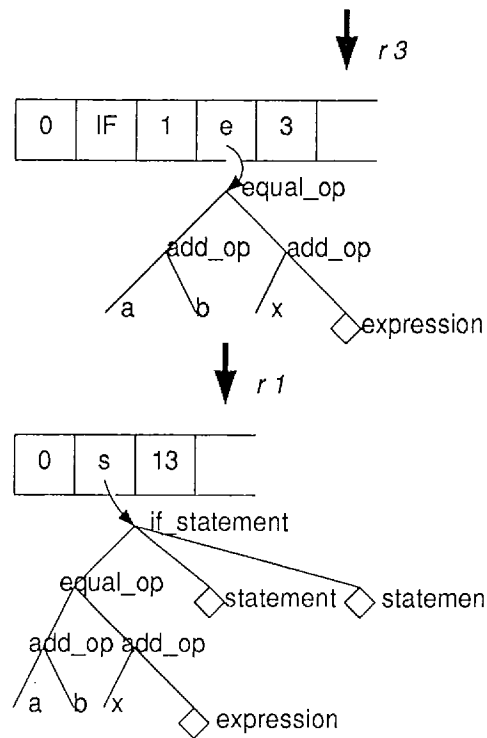
This automaton gives rise to the following SLR(1) tables (the *goto* table is not shown, since it is standard; “s” stands now for “shift”, “a” for “accept” and “r n” stands for “reduce with production no. n”):

<i>action</i>	IF	THEN	ELSE	+	=	t	\$	<i>handle_length</i>	<i>placeholder_no</i>
0	s								
1						s	r 1	1	3
2		r 2			s		r 2	1	
3		s					r 1	2	2
4	s						r 1	3	2
5			s				r 1	4	1
6	s						r 1	5	1
7			r 1				r 1	6	0
8		r 4			r 4		r 4	1	
9						s	r 3	2	1
10			r 3				r 3	3	0
11					s		r 5	2	1
12			r 5		r 5		r 5	3	0
13							a		

All ambiguities concerning incomplete input can be resolved by preferring reduction items over shift items. The resulting additional reductions on end-of-input are shown in italics in the action table. The *placeholder_no* is defined only if the corresponding tree building action actually builds a node and not just copies a pointer.

We will use the augmented parse table to process the example from section 2, “IF a+b=x+”. Upon end of input, the modified parser will perform the following additional incomplete reductions, starting with the configuration from section 2:





The modified parser behaves exactly as intended; it produces placeholders for missing textual input and finally reaches the “accept” state.

The method described in this paper is currently incorporated into the PSG system, a generator for language-specific programming environments [BS86]. PSG up to now employed a multiple-entry LL(1) parser, which accepted incomplete input as well as placeholders, but posed severe problems with certain languages. We will use our C editor as a testbed, especially for the conflict resolution strategies described above.

Acknowledgements. I would like to thank the students of my course on language-specific programming environments, who encouraged me to write down an idea I had developed one rainy sunday afternoon.

5 References

- [ASU86] Aho, A. Sethi, R. and Ullman, J.: Compilers – Principles, Techniques and Tools. Addison Wesley 1986.
- [BS86] Bahlke, R. and Snelting, G.: The PSG System: From Formal Language Definitions to Interactive Programming Environments. ACM TOPLAS 8, 4 (October 1986), pp. 547-576
- [B89] Borrás, P. et al.: Centaur: The System. Proc. ACM Symposium on Software Engineering Environments, ACM 1989, pp. 14 – 24.
- [F84] C. Fisher et al.: The Poe Language-Based Editor Project. Proc. ACM Symposium on Practical Software Engineering Environments, ACM 1984, pp. 21 – 29.
- [RT89] Reps, T. and Teitelbaum, T.: The Synthesizer Generator. Springer Verlag 1989.